

Constella: Choosing Query Compute without Rebuilding the Index

Dylan Couzon (Qdrant)

October 2026

Abstract

Can query computation become a choice on each request while a document index stays fixed? Constella adds two compatible query encoders to a frozen Stella document space: Zero, a token lookup table, and Nano, a 34.5M transformer, alongside the original Stella query path. They retain 81.4% and 90.5% of Stella’s exact-search nDCG@10 on BEIR-15. On a laptop CPU, warmed encoding takes 44 microseconds, 2.25 ms, and 31.6 ms for Zero, Nano, and Stella. All three search the same populated Qdrant collections without rebuilding between encoder batches. Compatibility does not imply identical search cost: Zero loses more quality to a narrow HNSW search, and its approximately 62-fold encoding advantage over Nano becomes 2.2-fold including retrieval on one binary-quantized million-passage diagnostic, preserving each tier’s own exact quality within 1%. A separate controlled scoring experiment holds document sign codes fixed and retains query magnitudes. At 40 candidates, this raises Zero’s original-top10 coverage from 82.2% to 93.0% on FiQA and 90.1% to 96.8% on the 1M diagnostic, with larger absolute gains than the other tiers but no native latency claim. A closed-form table study across 26 teachers also finds that the strongest registered teacher yields the weakest registered table. These results connect selectable query compute with retrieval quality, search effort, and query precision; build accounting shows what constructing the family requires.

1. Introduction

A document index can be expensive to replace: every document must be encoded again before a new retrieval model can serve it. Query-side distillation allows a different deployment choice. Keep the document vectors fixed, and train cheaper query encoders to search their space. An application can then choose its query computation budget without choosing a different document index.

We study this choice through **Constella**, a family with one frozen Stella document encoder and two compatible query encoders. Zero tokenizes, looks up vectors, pools, and normalizes; it runs no transformer forward pass. Nano uses a 34.5M transformer. The original Stella query path remains available as a third tier. An application can assign a tier to an endpoint or select an available tier on each request. The index remains unchanged. This makes compute a query-side decision, rather than requiring a separate dense index for each of these three budgets.

The construction has prior art [QED, EmbedDistill, pyNIFE, LEAF]. Our research question is **what this flexibility buys, and what limits it, once the document vectors are fixed**. Encoding benchmarks answer only part of that question. A cheaper query representation changes retrieval quality and can change how hard the same document graph is to search.

The principal finding connects those costs. Zero encodes 5-12-word queries in 44 microseconds on the measured CPU, retaining about four fifths of Stella’s BEIR-15 quality; Nano retains about nine tenths at 2.25 ms. Both search the same document vectors as Stella. But at a narrow HNSW setting on our 1M diagnostic, Zero loses 16.3% of its exact score, versus 7.0% for Nano and 4.4% for Stella. Retuning a shared binary index preserves useful speed differences; an unquantized index can absorb nearly all of Zero’s advantage over Nano. Index compatibility and search effort are separate properties.

A second finding separates query computation from query precision. Default one-bit scoring discards query magnitudes as well as document magnitudes. Keeping the document codes fixed, graded query scoring

restores 10.8 percentage points of Zero’s original-neighbor coverage at 40 candidates on FiQA and 6.7 on the 1M diagnostic. All tiers improve; Zero’s larger absolute gain also has more error headroom. The controlled result motivates measuring query precision separately, even when the query encoder runs no neural network.

The tiers also disagree on individual queries. A relevance-label oracle can match always-Nano while sending only 15% of queries to Nano on the 12 non-held-out datasets. Our simple frozen routers do not approach that oracle. Shared vectors make selective compute possible; choosing it well remains an open problem. The default tier can still be chosen by an application’s latency and quality requirements without an automatic router.

Two supporting results concern constructing such a family. Under one closed-form table recipe, a weaker teacher produces a much stronger table than the strongest registered teacher; the teacher’s retrieval score is a poor guide to that representation. This informs teacher choice before indexing, not swapping arbitrary teachers into an existing index. We also separate teacher-target preparation from fitting: Nano’s final optimization takes 57.3 hours on one A100, about \$95 at the historical rental rate, while the complete preparation and research effort costs more.

The contributions are the measured shared-index quality and compute choices, the query-dependent ANN penalty, a fixed-document-code query-precision control, and a bounded cross-teacher table comparison with explicit build accounting. Public model artifacts, a runnable inference quickstart, and the full evidence trail make the study inspectable. We do not claim a new architecture or a production routing policy.

2. Related work and contribution

Distilling queries while keeping documents fixed. QED trains a small query encoder by embedding alignment to a frozen document model [QED]. EmbedDistill studies geometric distillation, including asymmetric students [EmbedDistill]. LEAF explicitly targets modest data and infrastructure for teacher-aligned encoders [LEAF], and NanoVDR distills the text query path of a visual-document retriever [NanoVDR]. These works establish that query-side training can be practical; approachability alone is not this paper’s novelty.

Replacing the query network with a table. Model2Vec and static sentence embeddings average token vectors [Model2Vec, StaticEmb]. pyNIFE fits a lookup table against a frozen off-the-shelf teacher, reuses its document index, and proposes switching between a static and a contextual query path [pyNIFE]. LightRetriever trains a lookup-table query side jointly with an LLM document side [LightRetriever]. Our table architecture follows this prior work. We hold the document side fixed, compare one table recipe across many checkpoints, and measure its behavior inside an approximate-search engine.

Teacher choice and retrieval decisions. A stronger teacher need not make a stronger student in knowledge distillation [Cho and Hariharan 2019], including dense retrieval [PROD]. NanoVDR finds teacher quality predictive of student retention across datasets for one fixed teacher [NanoVDR]; our study varies teachers under one table recipe on the same datasets. Those are different comparisons. Retrieval strategy selection and query performance prediction also have prior work [Arabzadeh et al., QPP]. Compatible representation learning addresses index reuse when retrieval models change [BCT, FCT]. Our routing experiments are a bounded study of two compatible query encoders, not a claim to invent adaptive retrieval.

Query distributions and approximate search. OOD-DiskANN and RoarGraph establish that queries differing from the indexed-data distribution can make graph search harder; both use query information when constructing an index [OOD-DiskANN, RoarGraph]. QA-Cos improves binary-sketch candidate selection with query-aware decoding [QA-Cos]. Our work does not introduce OOD search or magnitude-aware scoring. We measure compatible encoder substitutions and a query-precision control over fixed text document representations.

Our empirical contribution is to measure what interchangeable query representations buy over fixed document vectors: exact quality, encoding cost, approximate-search work, and the candidate information lost to query precision. The cross-teacher table study and build accounting explain design and construction decisions behind such a family. We do not claim a new distillation architecture, a universal teacher-selection rule, or a generally superior small model.

3. Constella: one document index, three query tiers

3.1 The compatibility contract

The document encoder is NovaSearch/stella_en_400M_v5 at revision ffeb2b7e: normalized 1024-dimensional vectors, fp32 encoding, fp16 storage, cosine similarity. A replacement query encoder must align with that space. Matching vector width alone is insufficient; prompts, pooling, tokenization, normalization, and training targets matter. Zero and Nano were trained against this frozen space.

Query path	Computation per query	Construction
Stella	the original 400M transformer with its query prompt	original query path, no student to build
Nano	a 34,540,672-parameter transformer	bge-small layers 12, 8, and 4; linear projection to 1024 dimensions and mean pooling; squared L2 alignment to Stella targets
Zero	token lookup, count-saturated pooling, normalization	30,522 token rows trained with cosine and ranking losses, exported as int8

By **hot-swappable queries**, we mean selecting an available, aligned query encoder and sending its vector to the same collection. The stored document vectors need no re-embedding, and the three dense tiers need no separate document indexes. This is a compatibility capability, not a zero-time model-loading guarantee. E2 verifies all three paths against the same populated disk-backed Qdrant collection, without a collection update or rebuild between encoder batches, on FiQA and the 1M diagnostic. Queries were pre-encoded, then searched in batches; we did not benchmark concurrent request switching or production failover. The inference quickstart in README.md shows Nano and Zero using one collection. Appendix E records the prompt, tokenizer, and dtype pitfalls.

3.2 Available quality and compute budgets

Table 1. BEIR-15 macro exact-search nDCG@10 over the same Stella document vectors, with encoding p50 for 100 real 5-12-word queries, batch one, four CPU threads, Apple M5 Pro (ONNX Runtime; Zero on NumPy). Assets include model and tokenizer files. Registered E1 protocol; BEIR-15 aggregates are descriptive (results/m15_e1_latency.json, results/m20_beir15_run.json). Retention is a ratio of macros.

Query path	nDCG@10	Retention	Encode p50	Assets
Stella	0.5614	1.000	31.6 ms	1669.6 MiB
Nano	0.5081	0.905	2.25 ms	132.3 MiB
Zero	0.4572	0.814	0.044 ms	90.1 MiB

Nano keeps nine tenths of Stella’s quality at one fourteenth of its encode latency; Zero keeps four fifths at one seven-hundredth. Zero’s no-transformer path makes encoding almost negligible in the measured warm case. It is not zero retrieval latency, zero cold-start cost, or zero memory: Zero takes 0.215 s to hydrate and 0.208 ms for its first query under E1. The asset totals are distinct from peak process memory; the full E1 inventory is in m15/EVIDENCE.md. All-tier resident memory was not measured.

These are different quality points, not equal-quality speedups. Retention varies by task: Zero retains 0.667 on TREC-COVID, while Nano retains 0.988 on Quora. Zero exceeds BM25 on 11 of 15 datasets descriptively (BM25 macro 0.4006); the original confirmatory BM25 contrast did not pass its correction. Nano scores below bge-small on bge-small’s own index (0.5171) and LEAF on its arctic-embed-m index (0.5402 at 1.39

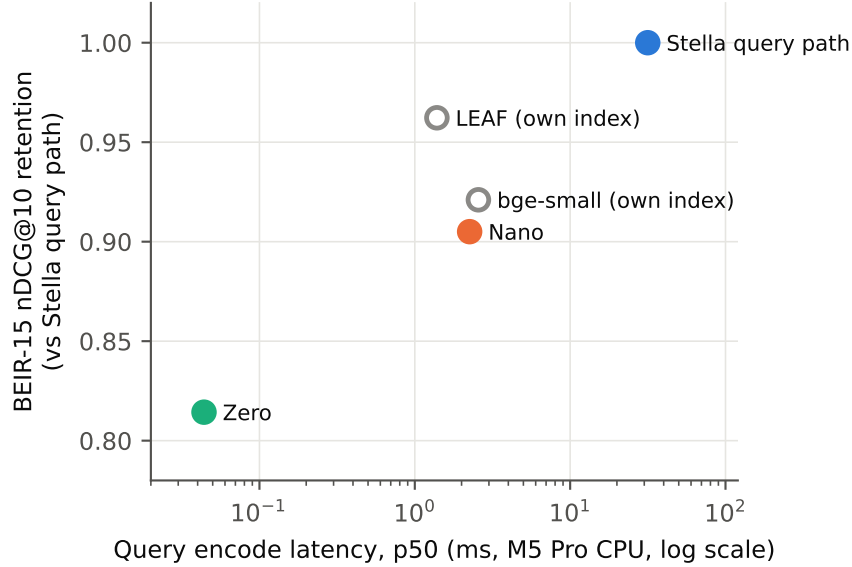


Figure 1: BEIR-15 retention against encode latency. Hollow markers are reference systems on their own indexes (bge-small 0.5171 at 2.57 ms, LEAF 0.5402 at 1.39 ms).

ms). A team building a new index should consider those alternatives. Constella’s distinguishing deployment capability is keeping these three query paths compatible with the Stella document vectors.

3.3 What selective compute makes possible

Compatible vectors allow per-request tier selection and a common-index reference or fallback. They do not establish that the most expensive tier wins every query. On the 12 datasets outside the held-out test, Zero is better than Nano on 18% of queries and Nano on 39%, averaging dataset shares. A label-aware oracle reaches 0.5473 macro nDCG@10 against 0.5161 for always-Nano, and matches always-Nano using Nano on only 15% of each dataset’s queries (`results/m15_e5_oracle.json`, registered). The tiers used different training pools, including Zero’s FEVER-family exposure, so these differences are not a matched-data architecture result.

That oracle is headroom, not a working router. Three query-only signals recover little of its gain. Zero’s retrieval-score margin recovers about a quarter on the six public sets, but requires a first search and a second search for escalated queries. Appendix C gives the frozen policies, costs, and a vector blend that failed to transfer. The evidence supports configurable query budgets now; an effective adaptive policy remains a research opportunity.

4. Changing the query encoder changes the search

4.1 Encoding savings and graph-search work

A table’s query needs a wider graph search. We served each tier from Qdrant (v1.19.1, native macOS build) over a one-million-passage MS MARCO subset that keeps every judged positive and fills the rest at random (a diagnostic, not full MS MARCO; MS MARCO is used for validation only), and over FiQA (57,638 documents). For each tier, we swept HNSW `ef` and three quantizations (int8 scalar, 1-bit binary, and 4-bit TurboQuant, each with rescoring and 1x-4x oversampling). We selected the cheapest end-to-end setting within 1%, 2%, and 5% of that tier’s own exact nDCG@10 (registered method). Every segment was HNSW-indexed before timing (`indexing_threshold` set to 1 KB instead of the 10,000 KB default, so no vector was served by a plain scan); this is a benchmark setting, not a recommendation. At the same unquantized setting on the 1M collection (`ef` 16), approximate search costs Zero 16.3% of its exact score, Nano 7.0%, and Stella 4.4%. Their top-10 lists agree with exact search on 82%, 91%, and 93% of neighbors

(results/m15_e2_ann_msmarco1m.json). Median cosine to the nearest document is 0.60 for Zero, 0.74 for Nano, and 0.78 for Stella. The table’s averaged query vector sits farther from documents used to build the graph, consistent with, though not a test of, a harder graph search.

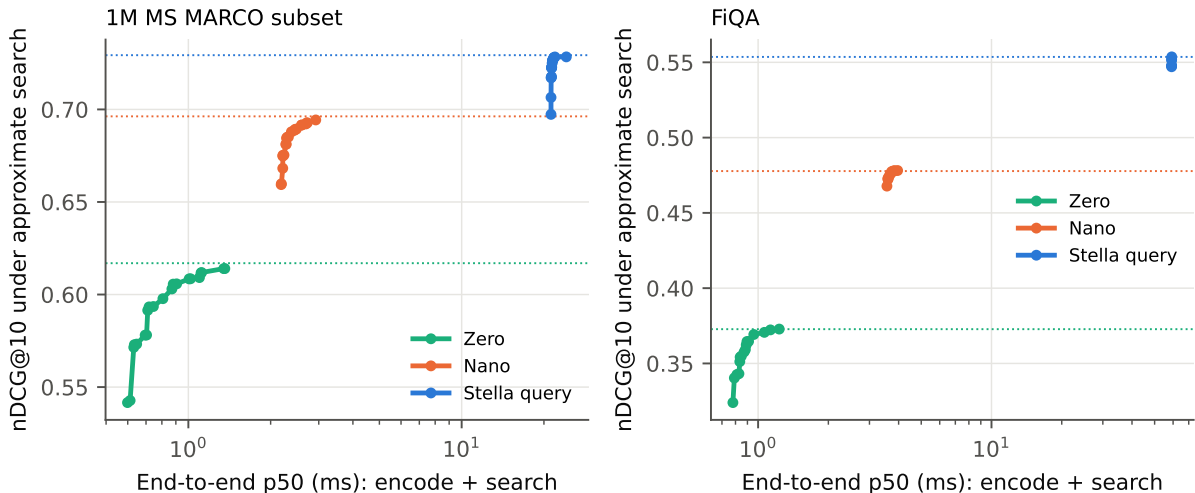


Figure 2: Approximate-search nDCG@10 against encode-plus-search p50. Each point is the fastest recorded configuration reaching that quality; configurations can use different quantization collections, with all three query paths measured on each collection. Dotted lines mark each tier’s exact score. Left: 1M MS MARCO diagnostic; right: FiQA.

Three costs on one physical index. A literal shared-index example comes from the binary-quantized 1M collection. Keeping each tier within 1% of its own exact score, Zero uses request ef 512 with 2x rescoring oversampling, Nano ef 256 with 4x, and Stella ef 128 with 1x. Their nDCG@10 scores are 0.6116, 0.6893, and 0.7226, at encode-plus-search p50 of 1.11, 2.48, and 21.3 ms. The documents, graph, and quantization stay fixed; query vectors and request search settings change. This is an exploratory illustration from the registered sweep, preserving each tier’s own quality rather than equal absolute quality. The approximately 62-fold Nano/Zero encoding gap on these short queries becomes 2.2-fold after retrieval.

Search configuration can absorb the saving. On the unquantized 1M collection, the fastest recorded settings within 1% of each tier’s exact score take 3.61 ms for Zero and 3.85 ms for Nano. Here the additional graph work nearly consumes Zero’s encoding advantage. Allowing a different quantization configuration for each tier gives a Nano/Zero ratio of 2.2 at 1% and 3.1 at 5%. FiQA gives 3.8-4.3 across those targets (results/m15_e2_ann_fiqa.json). These are configuration-specific findings, not a universal encoder speedup. Figure 2 shows the full configuration frontier. Encode-plus-search is the sum of per-query times measured in separate phases of the same process, excluding model loading and application dispatch.

Lexical fusion spends another part of the budget. Zero + BM25 improves its BEIR-15 macro from 0.4572 to 0.4933 (87.9% of Stella); Nano + BM25 reaches 0.5110 (91.0%). This is an additional sparse retrieval channel, not another dense encoder swap. On the 1M diagnostic, server-side BM25 plus dense prefetch of 100 and DBSF fusion takes 4.71 ms including Zero encoding, with nDCG@10 0.6183. That score belongs to the diagnostic, not BEIR-15. The dense-only binary example above takes 1.11 ms at 0.6116; fusion uses a separate unquantized dense-plus-sparse collection. Fusion buys quality with additional search and index work, even when encoding is almost negligible.

The engineering implication is to measure search again after a query encoder changes, even when the document vectors and graph are unchanged. The table’s exact quality loss and its approximate-search loss are distinct. The speedup remaining after retuning is still useful on the measured workload, but it is much smaller than its encoding speedup.

4.2 Query precision is a separate budget

A same-collection diagnostic. E2’s precision configurations used separate graph builds. To isolate native search scoring, we ran 192 deterministically selected queries on one fixed binary collection per workload, switching original-vector scoring (`ignore=true`), binary scoring, rescoring, and oversampling (`results/m15_e16_quantization_pilot.json`, `results/m15_e17_quantization_replication.json`, `exploratory`). At the primary `ef` 64, binary scoring with rescoring and oversampling 1 loses 11.7 points of exact-neighbor recovery for Zero on FiQA, 5.1 for Nano, and 2.3 for Stella. The 1M replication is weaker: 3.9, 3.1, and 2.2 points, with extra Zero-versus-Nano uncertainty including zero. Secondary `ef` settings do not replace that primary result. Native rescoring also changes collection-level candidate selection through segment merging; these treatments are not a pure rerank of one fixed global pool.

Hold document codes fixed and change only query precision. The tagged Qdrant 1.19.1 implementation defaults to sign-coded queries for one-bit documents [Qdrant 1.19.1]. We separately compare exhaustive `sign(q)` dot `sign(d)` scores with original normalized `q` dot the same `sign(d)`, using the identical 192 query vectors and each tier’s original exact-top10 target. Global candidate budgets are 10, 40, and 100; 40 is the primary display. Coverage is expected target-neighbor inclusion under uniform selection at boundary ties (`results/m15_e18_query_precision.json`, `exploratory`). No graph or new encoder is used.

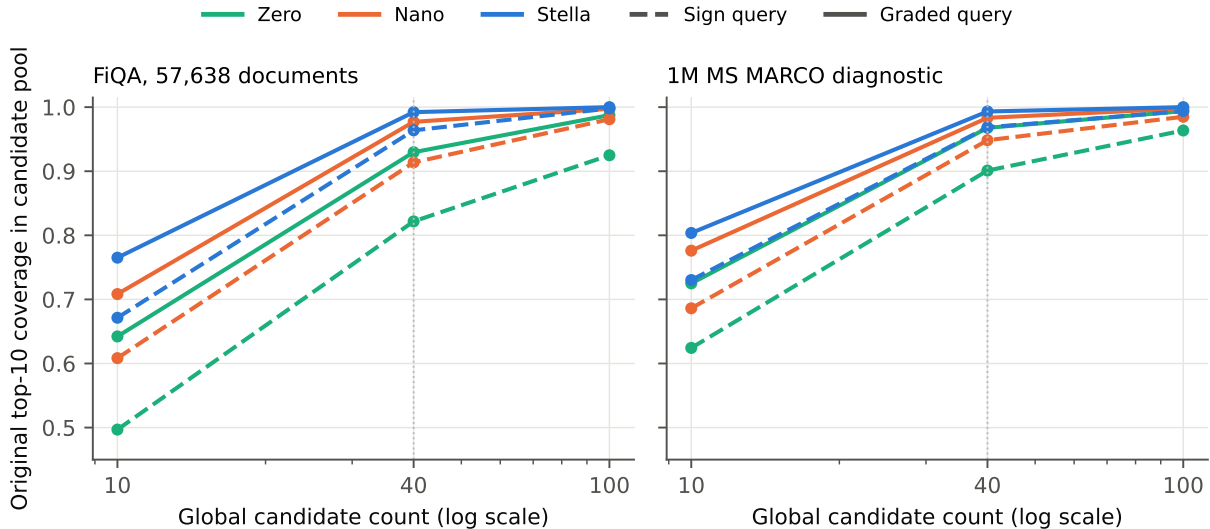


Figure 3: Exact candidate selection on fixed one-bit document codes. Each tier retains its own original-vector top10 target; solid lines retain query magnitudes, dashed lines keep signs only. Coverage averages expected inclusion under boundary ties over 192 queries per workload; 40 candidates is the primary display. This is an exact-scoring control, not a native ANN or latency benchmark.

At 40 candidates, Zero’s coverage rises from 0.8217 to 0.9297 on FiQA and from 0.9010 to 0.9677 on the 1M diagnostic. Nano gains 6.33 and 3.49 percentage points, and Stella 2.84 and 2.47. The extra absolute Zero gain versus Nano is 4.46 points on FiQA (95% paired query interval 2.53-6.43) and 3.17 on the 1M diagnostic (1.58-4.75). Larger absolute gains have more headroom: Zero does not recover a larger fraction of its sign-mode misses than the other tiers. Mean cosine to the sign-query direction is about 0.80 for every tier. Zero does not show a larger mean angular distortion; the source of the coverage difference remains unresolved. Full budget rows and miss fractions are in the evidence file.

Preserving query magnitudes recovers useful candidate information without changing the document codes or the encoder. Query precision therefore remains a separate choice even when encoding is almost negligible. The graded scorer is an analytical control, not native 8-bit query encoding; global candidate budgets are not per-segment oversampling, and candidate coverage is not nDCG or serving latency. Validating the supported native precision setting and its cost is the next deployment test.

5. Building compatible query encoders

5.1 Preparation is separate from fitting

A query-only alignment objective can learn from query text and cached teacher query vectors. Ranking losses also need the appropriate document vectors or training pairs. None of this requires rebuilding an already compatible serving index, but the training targets still have to be produced. We separate three stages: selecting and preparing permitted data, encoding teacher targets, and optimizing or solving for the student.

Table 2. Component build costs. Nano’s time is measured; its dollar value prices that time at the recorded historical rental rate. Zero’s figures are approximate historical ledger estimates. Sources: `results/m15_e15_decision_audit.json`, `results/m13_build_record.json`, `results/m13_build_allocation.json`, `m7/LEDGER.md`.

Artifact	Fitting or optimization	Preparation and accounting boundary
Closed-form probe	4-7 minutes per registered configuration on an A100 for fitting and development scoring, including fit-query encoding except for cached Stella targets	coarse intervals between output files; downloads, data assembly, and index construction excluded
Zero	approximately 20 minutes of retraining on an RTX 3080, historical estimate	changing teachers was estimated to require 8-12 hours of target re-encoding; not a measured complete rebuild
Nano	57.3 A100 hours for 199,999,721 example presentations; about \$95 at the recorded \$1.664/hour rental rate	teacher targets already prepared; excludes data generation, preparation, recipe search, failed runs, evaluation, export, and engineering time

The Nano figure shows that the final student optimization did not require a cluster. It does not show that the entire project can be reproduced from scratch for \$95, or that this training dose is the cheapest way to reach its quality. The table timings likewise show that fitting rows after teacher encoding can be short; they do not make the encoding stage free.

Zero uses project-approved sources including ESCI, FEVER-train, HotpotQA-train, Mr. TyDi English, and SQuAD-train, plus NQ-open and TriviaQA query text (`m7/RECIPE.md`). Nano uses a different pool and a 75% query / 25% document batch mix (`m13/build_config.json`). MS MARCO is validation-only in our distillation, and Nano’s pretrained bge-small backbone has its own MS MARCO exposure. Reusing an index does not resolve training-data permissions.

5.2 Teacher choice under one table recipe

If an index already exists, its document encoder is fixed: the design question is whether an intended cheap representation retrieves well in that space. Teacher selection becomes available only before building an index. We study that decision separately with a **closed-form table probe**, not the trained Zero or Nano recipe. The probe uses ridge regression from token counts to teacher query vectors, anchored at teacher-derived token rows. It can itself be a candidate query encoder; its quality does not certify another student recipe.

We fit one closed-form table recipe to 10 checkpoints from six model families, all sharing a BERT WordPiece vocabulary. The fit list contains 337,981 queries screened against the held-out data. Each ridge weight was selected on CQADupStack physics and programmers, then frozen before the six public BEIR sets were scored (`results/m15_e8_frozen_lambdas.json`, `results/m15_e8_towers.json`, `registered`).

After observing that result, we added 17 checkpoints under the same recipe and freeze order: MiniLM, bge, e5, thenper gte, arctic, Contriever, and TAS-B (`results/m15_e8x_towers.json`, exploratory). One arctic checkpoint failed the convergence gate and was excluded, leaving 26. Every teacher is evaluated over its own document vectors; this section compares teacher spaces, rather than swapping teachers into the Stella index.

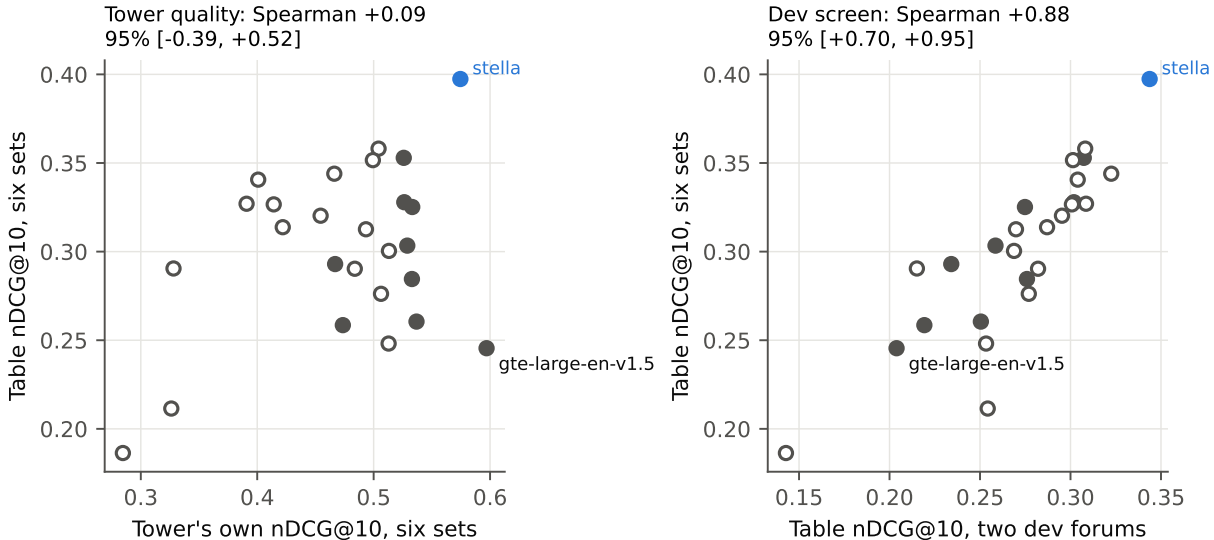


Figure 4: One closed-form recipe across 26 checkpoints. Filled points are the registered ten, hollow points the exploratory 16. Teacher quality weakly ranks table quality; directly evaluating the table on development queries ranks it much better. Intervals are 10,000-draw bootstraps over checkpoints.

Across 26 checkpoints, teacher quality and table quality have Spearman correlation $+0.09$ (95% interval -0.39 to $+0.52$). The interval allows moderate associations; the evidence does not establish independence. The concrete reversal is clearer: gte-large-en-v1.5 is the strongest registered teacher and yields the weakest registered table. Stella shares its backbone, tokenizer, and 1024-dimensional architecture but differs in training and readout, and yields the best table across all 26.

Table 3. Teacher-choice consequences under the closed-form recipe, six-set macro nDCG@10. Teacher and table scores use the same six datasets and each teacher’s own document index. Registered rows; the selection difference is an exploratory derivation in `results/m15_e15_decision_audit.json`.

Teacher	Teacher score	Table score	Table / teacher
gte-large-en-v1.5	0.5970	0.2455	0.411
Stella	0.5745	0.3974	0.692

The development screen chooses Stella. Selecting the strongest public teacher in hindsight would choose gte-large, so the screen’s selected table is higher by 0.1519 nDCG@10. This illustrates the cost of choosing by teacher quality even with the teacher’s public score available; it is not a prospectively tested model-card selection policy.

The table’s development score ranks its public score with Spearman 0.88 across the 26 (95% interval 0.70 to 0.95), against 0.90 across the registered ten. The family-resampling sensitivity check is in `results/m15_e14_screen_checks.json` (exploratory). The result supports directly evaluating this cheap representation instead of inferring its quality from the teacher.

Stella wins the six-set table macro, but the development ranking agrees less strongly with the four sets without disclosed Stella overlap (Spearman 0.68). On those four, the selected table trails the best of the

registered ten by 0.0048 and the best of the pooled 26 by 0.0278. Across the 26, it ranks fifth on SciFact and eleventh on TREC-COVID (`results/m15_e15_decision_audit.json`, exploratory). A screen’s development workload must reflect the quality objective; these forums do not select the best table for every domain.

The effect is not explained by three exploratory vector-fidelity diagnostics; Appendix D.3 preserves those and the dimension associations.

The scope is one table recipe, one shared vocabulary, and six public text-retrieval datasets. Most teachers trained on related source families; exposure for the registered ten is disclosed in `m15/e8_exposure.json`, while the exploratory 16 were not individually audited. An earlier project fit list had 1.31% held-out-query overlap; this comparison uses its cleaned successor. No result here demonstrates that the screen predicts the separately trained Zero recipe or transformer students.

5.3 Using the family and building another one

Using the existing models and training a new compatible encoder are different tasks. The inference quickstart selects Nano or Zero over one collection, with model identities and serving parity documented in the release records. It requires no student training. The recipes expose how the released students were built; a standalone bring-your-own-teacher training tool is not part of this release.

For a fixed index, fit and evaluate the intended cheap representation against its actual document vectors. If selecting a teacher before indexing, directly evaluate candidate cheap query paths rather than relying on teacher quality. Freeze choices on representative development queries, keep the original query path as a reference, and evaluate on other queries. Record target preparation and fitting separately, then retune approximate search for each tier. The probe driver (`m15src/e8_towers.py`), Zero recipe (`m7/RECIPE.md`), and Nano build configuration (`m13/build_config.json`) provide concrete starting points, with the scoped evidence above rather than a promise of a complete \$95 reproduction.

6. Evaluation boundaries and limitations

The exact quality rows use 15 BEIR datasets (`results/m20_beir15_run.json`). Four datasets were held out for one registered test: FEVER, DBpedia-entity, and CQADupStack android and english. Their published aggregate results enter BEIR-15 but play no fitting or diagnostic role. Ridge weights, router thresholds, and blend weights use the other two forums. The trained tiers were built earlier on the project’s development suite.

The teacher, Zero, and Nano are not matched-data architecture ablations. Zero trained on FEVER-train and HotpotQA; Nano’s pool excluded FEVER. Zero’s higher scores on FEVER-family datasets therefore mix architecture with data. Our single-prompt Stella path scores 0.5614 on BEIR-15, below its model card’s 58.97 under task-specific instructions, a 400-token limit, and bf16. These results characterize our pinned serving path. Appendix B preserves the registered comparator contrasts and all four held-out aggregates, including unresolved and negative outcomes.

We trained the served recipes against one teacher. The broader teacher comparison tests closed-form tables only, with 16 of 26 checkpoints added after the registered observation. Neither teacher-rank correlations nor checkpoint bootstraps establish a general law of distillation. BEIR-15 and the new cost and selection audit are descriptive; query-resampling intervals exclude training-seed variation.

Deployment measurements use one laptop CPU, one runtime, and two collections. We did not measure server throughput or GPU serving. The MS MARCO subset retains judged positives while removing most competing passages, so its absolute nDCG@10 is optimistic. Its search penalties concern that subset. End-to-end latency ratios preserve each tier’s own quality and do not establish equal-quality superiority. Build costs price recorded compute and estimates; they omit the complete research and engineering effort. All-tier resident memory, encoder-load switching overhead, and concurrent dispatch were not measured.

The new precision studies use 192 selected queries per workload and fixed trained models. Native option contrasts are workload-sensitive, and the 1M primary interaction is weaker than FiQA. E18 isolates sign

versus graded query scoring on fixed document codes; it does not evaluate native scalar8 performance, graph traversal, or relevance gains. Absolute candidate gains partly reflect different error headroom, not an architecture sensitivity law.

A symmetric small model is also a real alternative. Nano scores below bge-small and LEAF on their own indexes in our BEIR-15 comparison. Its practical value depends on preserving an existing Stella index and offering compatible query paths. We do not measure the full cost of migrating to another index, so index reuse is a deployment constraint and capability, not a quantified savings claim.

7. Conclusion

Constella makes query compute selectable over one frozen Stella document index: a no-transformer lookup table, a compact transformer, and the original query path offer distinct quality and cost budgets. The nearly negligible table encoding does not make search free. Changing query vectors changes the work needed to traverse the same document graph, so an encoder benchmark cannot determine the deployment saving. Shared-index compatibility also exposes routing headroom, although the simple policies tested here fall well short of the label-aware oracle.

Query compute and query precision are separate budgets. With document sign codes fixed, preserving query magnitudes improves original-neighbor candidate coverage for all three tiers, with substantial absolute gains for Zero. This gives a controlled reason to test search precision when adding a cheap query path; the native cost and geometry mechanism remain open.

Building such a family requires evaluating the intended cheap representation, not only its teacher. The strongest registered teacher produces the weakest table under our closed-form recipe, and development screening transfers much better, with meaningful task sensitivity. Fitting can be approachable on one GPU, but teacher-target preparation and research are separate costs. Together, these results make the design concrete: keep the documents fixed, choose query computation deliberately, and measure quality and search effort for each available path.

Appendix A. What a mean-pooled table absorbs

The full proof is in `m15/PROOF_ABSORB.md`. Let a table with rows w_t pool a query with weights $\alpha_t(q) \geq 0$ that sum to one (mean pooling, and Zero’s count-saturated pooling), then normalize. (1) Applying $x \mapsto Ax + b$ after pooling equals pooling the table $w'_t = Aw_t + b$, because $\sum_t \alpha_t(Aw_t + b) = A \sum_t \alpha_t w_t + b$; under sum pooling the offset would scale with query length instead. (2) Reweighting tokens by fixed $g_t > 0$ and renormalizing equals the table $g_t w_t$ up to a positive scalar that the final normalization removes. Centering, whitening, principal-component removal, IDF or SIF weighting and their compositions are therefore tables of the same shape. The lemmas do not cover nonlinear maps, pooling weights that depend on count patterns, or word order.

Appendix B. Registered evaluations

Contrast	Partition	Difference in nDCG@10	Registered outcome
Nano - bge-small	clean-4	+0.017648 (one-sided lower bound +0.003674)	established
Nano - bge-small	all six	+0.027449	established
Nano - LEAF	all six	+0.016181	established
Nano - LEAF	clean-4	-0.001063	unresolved
Nano - bge-small	held-out NDO-3, dataset-weighted	+0.0032 [-0.0069, +0.0134]	registered descriptive
Nano - bge-small	held-out NDO-3, query-pooled	-0.0121 [-0.0219, -0.0024]	registered descriptive
Nano - LEAF	held-out NDO-3	-0.0389 [-0.0488, -0.0290]	registered descriptive

Clean-4 is NFCorpus, SCIDOCs, SciFact and TREC-COVID, the six-set members Stella’s recorded training data does not name. NDO-3 weights DBpedia-entity 0.5 and the two held-out CQADupStack forums 0.25 each; FEVER’s row is in `m15/EVIDENCE.md`. “Unresolved” means the registered rule did not establish a difference; no equivalence interval was computed. Sources: `m21/BENCHMARKS.md`, `results/m13_reserved_run.json`.

The originally held-out four, copied only from the published aggregate receipt `results/m13_reserved_run.json`:

System	FEVER	DBpedia	CQADup android	CQADup english
nano-dense	0.6231	0.4190	0.4774	0.4298
zero-dense	0.6978	0.3900	0.4431	0.3690
stella-query	0.8207	0.4603	0.5275	0.5183
bge-small-en-v1.5	0.8662	0.4003	0.4761	0.4558
leaf-ir-asym	0.8671	0.4520	0.5260	0.4707
bm25	0.5036	0.3045	0.3952	0.3481
zero+bm25 dbsf@100	0.7559	0.4030	0.4629	0.4016
nano+bm25 dbsf@100	0.6879	0.4296	0.4750	0.4323

Zero’s original registered dense test missed the LightRetriever-derived bar. Its comparison with BM25 failed the Holm threshold despite a positive unadjusted interval; its fused comparison with the OpenSearch reference was unresolved (`m21/BENCHMARKS.md`). These outcomes remain part of the research record and establish neither superiority nor equivalence. Full six-set and BEIR-15 rows for every system are in `m15/EVIDENCE.md`.

Appendix C. Routing and vector blending

C.1 Routing headroom

On the 12 BEIR-15 datasets outside the held-out test, averaging per-dataset shares, Zero and Nano both score 0 on 17% of queries, tie above 0 on 26%, and Zero is higher on 18% and Nano on 39% (pooling all queries gives 56% ties, because Quora is large and tied on 72% of its queries). A per-query oracle with relevance labels picks the better tier and reaches 0.5473 macro nDCG@10, against 0.5161 for always-Nano and 0.5580 for Stella. Sending only 15% of each dataset’s queries to Nano, largest gains first, already matches always-Nano; at 25% it reaches 0.5334 (`results/m15_e5_oracle.json`, registered). Without Climate-FEVER, the one FEVER-family set among the 12, always-Nano scores 0.5406 and the oracle needs 20% of queries to match it (0.5453). This label-aware oracle bounds what routing can gain.

C.2 Frozen routers

A router must decide before running Nano. The signals are subwords per word, pooled-vector norm before normalization, word count, the gap between Zero’s first and tenth document scores, and the number of documents shared by Zero and BM25 in their top ten. For each signal, we fitted its direction and threshold on the two development forums at Nano budgets of 10%, 25%, and 50%, then froze them. We compared each router with random routing at its realized Nano share and with the oracle at that share (`results/m15_e6_router.json`, `results/m15_e10_router.json`, registered).

Signal	Available after	Gain over random, 25% budget	Oracle gain share	Sets
Fertility	encoding	+0.0049	0.10	12
Pooled vector norm	encoding	+0.0016	0.03	12
Word count	encoding	+0.0018	0.04	12
Zero score margin	Zero search	+0.0175	0.25	6
Zero/BM25 overlap	both searches	+0.0154	0.31	6

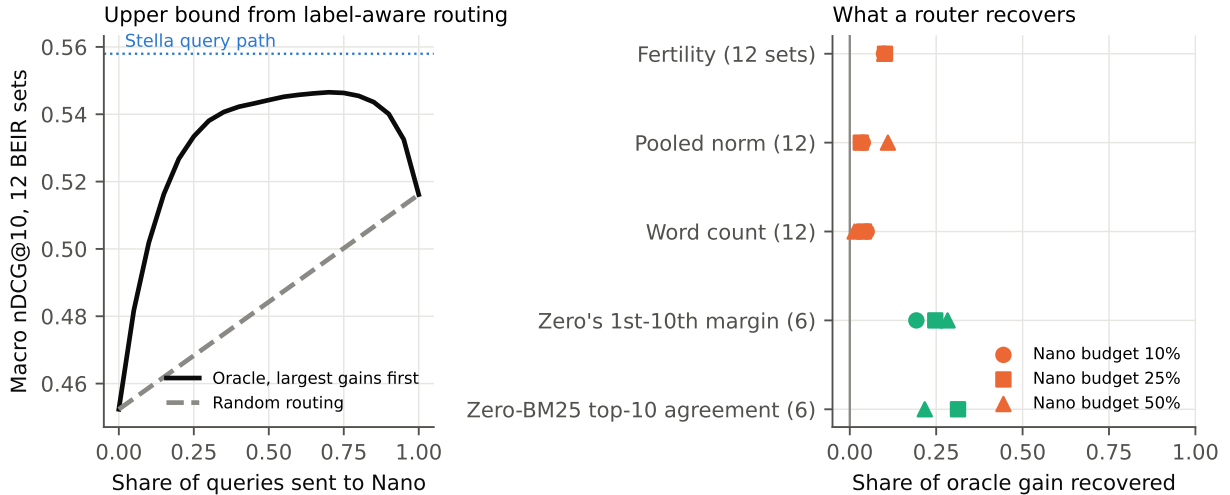


Figure 5: Left: the upper bound from label-aware routing on the 12 evaluation sets, against random routing. Right: the share of the oracle’s gain over random routing each frozen router recovers, per Nano budget; orange signals are known before the search, green ones after Zero’s own search.

The three query-only signals carry little. Zero’s retrieval margin carries more: sending queries with the narrowest margin to Nano (28% of each dataset’s queries on average, 24% pooled) scores 0.4865 on the six sets, against 0.4690 for random routing at that share and 0.5317 for always-Nano. Gains are largest where Zero is weakest: +0.043 on SciFact and +0.032 on TREC-COVID. Sending routed queries to the Appendix C.3 blend instead of Nano scores 0.4834, 0.0031 lower. The agreement signal counts shared documents; at the 10% budget its frozen threshold routes no queries.

C.3 Cost and vector blending

A margin router first searches with Zero, then encodes and searches again for escalated queries. Its cost therefore includes two searches on those queries. Section 4’s modest end-to-end gap limits the latency it can save on this workload; the oracle is a relevance upper bound, not a deployable speedup.

Zero and Nano write into one space, so a system can search once with $\text{normalize}((1 - a) \text{Nano} + a \text{Zero})$, adding 0.044 ms to Nano. The development forums chose $a = 0.3$, raising Nano there from 0.4247 to 0.4295. On the six public sets, the frozen blend lowered it by 0.0051 (95% interval -0.0092 to -0.0011), gaining only on ArguAna (+0.005) and losing most on TREC-COVID (-0.016). For Stella, the forums chose no blend (`results/m15_e9_blend.json`, registered descriptive; a pre-freeze code check printed SciFact’s curve, recorded in `m15/MEASUREMENTS.md`). The forums that ranked teachers in Section 5.2 did not select a useful blend weight.

Appendix D. Query and teacher diagnostics

D.1 Shuffle sensitivity

The unigram table is invariant to permutations of its token counts. On 3,727 queries of six public sets, the Stella-minus-Zero gap correlates with Stella’s own nDCG@10 drop under five word shuffles (Spearman 0.52). The 40% of queries on which shuffling hurts Stella carry about 90% of the observed net gap (`results/m15_e12_failure_modes.json`, exploratory). This localizes a failure pattern but does not identify its cause: both differences share Stella’s full-query score, shuffling changes more than word order, and positive and negative query gaps cancel in the net total.

Stratifying by broad bins of Stella’s score leaves the association at 0.53. A separate control perturbs Stella’s query vector once in an isotropic random direction at the same cosine distance as one shuffle. Among 3,069 queries with positive Stella scores, that perturbation costs 0.009 nDCG@10 on average, against 0.047 for shuffling (`results/m15_e12b_fragility.json`, exploratory). Broad score bins do not remove all shared-score coupling, and equal-distance isotropic noise does not match a language perturbation’s direction relative to ranking boundaries. These controls establish a bounded diagnostic, not that word order explains a particular fraction of the loss or that general fragility has been excluded.

Subwords per word correlate with the gap at 0.10. That feature measures fragmentation, not token rarity or training support. Two selected FiQA failures illustrate the diagnostic (`results/m15_e13_robustness.json`, exploratory). For “What credit card information are offline US merchants allowed to collect for purposes other than the transaction?”, the first relevant answer ranks first with Stella, 59th after shuffling, 11th with Nano, and outside Zero’s top 100. For “What can I do with a physical stock certificate for a now-mutual company?”, the corresponding ranks are 1, 8, 13, and 29. These queries were selected for large shuffle sensitivity and large Stella-minus-Zero gaps, so they do not represent the typical query.

D.2 Short and unfinished queries

We cut each SciFact, NFCorpus, and FiQA test query and measured the share of its own full-query nDCG@10 each tier keeps (macro over the three sets, registered, `results/m15_e4_prefix.json`):

Cut	Stella query	Nano	Zero	BM25
First word	0.293	0.288	0.297	0.362
First two words	0.389	0.381	0.387	0.448
First three words	0.479	0.468	0.461	0.528
First half of the words	0.610	0.597	0.604	0.656
First half of the characters	0.549	0.533	0.547	0.488

The dense tiers’ shares differ by at most 0.02 at every cut shown and by 0.034 at 75% of the words (0.846, 0.829 and 0.812 for Stella, Nano and Zero). This is a descriptive similarity on cut test queries, not an equivalence test or real typing. BM25 keeps a larger share on word prefixes and a smaller one when a cut lands inside a word.

D.3 Teacher diagnostics

Higher-dimensional checkpoints tend to produce weaker tables under this recipe. The association with absolute table quality is -0.44, while dimension correlates with teacher quality at +0.61. Its stronger association with retention, -0.76, partly reflects the teacher score in the denominator. Dimension is not parameter count and is confounded with training and readout; this is not a causal result about model size. The within-family observations and both absolute and relative correlations are recorded in the evidence file.

Vector fidelity is also an unreliable substitute for retrieval evaluation in these experiments. The e5 tables match their teacher query vectors more closely than Stella’s table does (mean cosine 0.90 and 0.89, against 0.78), yet retrieve worse. Three exploratory scalar diagnostics, including error relative to ranking margins, do not explain the teacher ranking reliably at $n = 10$ (`results/m15_e11_mechanism.json`, `results/m15_`

e11b_margin.json). We have established a practical reversal and a useful direct screen, not its underlying cause.

Appendix E. Quality spread and serving compatibility

Per-dataset BEIR-15 rows for all eight systems, E8’s per-dataset tables and the full E1 latency inventory are in m15/EVIDENCE.md.

A swap is legal when the replacement emits vectors aligned with the index’s document space, with the same width, normalization and similarity; our served paths match their references to 4.5e-08 (Zero), 1.2e-07 (Nano) and a minimum cosine of 1.00000000 (Stella through the published document graph). Three things fail silently: Stella without its prompt scores at cosine 0.80 against the correct vector; Stella’s tokenizer pads to 512 by default, which drops cosine to 0.35; and FastEmbed’s mean pooling returns float64 for mean-pooled models, Nano among them, which doubles memory and changes the raw bytes (qdrant/fastembed issue #752, recorded in the project audit; our measurements pool in float32 with ONNX Runtime directly).

Appendix F. Reproducibility

The inference quickstart in README.md builds one collection and queries it with Nano and Zero. Artifact identities are Qdrant/stella-en-400M-v5-doc-onnx, Qdrant/constella-zero, and Qdrant/constella-nano; pinned model hashes and implementation parity are recorded in the release milestones and E1.

Every figure regenerates from committed JSON (python m15/figures/make_figures.py) and every number in m15/EVIDENCE.md from python m15/make_evidence.py. M15 result files (results/m15_*.json), except E8’s frozen-lambda file (written on the pod and bound by hash into E8’s result), carry the script hash, git commit, seed and machine; model and dataset revisions are recorded in the result, in the scripts it names, or in the result it was derived from (E8’s frozen lambdas are bound by hash into E8’s result; E12 and E12b take their pins from the committed M20 rows). Earlier results carry the provenance records of the milestone that produced them. Methods and reviews: m15/MEASUREMENTS.md, m15/REVIEWS/.

E15 derives teacher-choice consequences and component build costs from five published JSON receipts and the Zero ledger, without new training or raw evaluation access. To check E15 without overwriting its committed receipt, write a new result to an unused scratch path:

```
.venv/bin/python m15src/e15_decision_audit.py \
  --output work/m15/e15-check/result.json
```

The script records input and code hashes; the paper distinguishes measured training time, priced component cost, and historical estimates.

References

Primary-source checks and supporting passages are recorded in m15/RELATED_WORK.md, m15/REVIEWS/2026-10-01-owner-literature.md, m15/REVIEWS/2026-10-01-followup-literature.md, and m15/REVIEWS/2026-10-01-quantization-semantics.md.

- [QED] Yuxuan Wang and Hong Lyu. Query Encoder Distillation via Embedding Alignment is a Strong Baseline Method to Boost Dense Retriever Online Efficiency. SustainNLP Workshop at ACL, 2023. [arXiv:2306.11550](#).
- [EmbedDistill] Seungyeon Kim, Ankit Singh Rawat, Manzil Zaheer, et al. EmbedDistill: A Geometric Knowledge Distillation for Information Retrieval. 2023. [arXiv:2301.12005](#).
- [LEAF] Robin Vujanic and Thomas Rueckstiess. LEAF: Knowledge Distillation of Text Embedding Models with Teacher-Aligned Representations. 2025, revised 2026. [arXiv:2509.12539](#).
- [NanoVDR] Zhuchenyang Liu, Yao Zhang, and Yu Xiao. NanoVDR: Distilling a 2B Vision-Language Retriever into a 70M Text-Only Encoder for Visual Document Retrieval. 2026. [arXiv:2603.12824](#).

- [LightRetriever] Guangyuan Ma, Yongliang Ma, Xuanrui Gou, Zhenpeng Su, Ming Zhou, and Songlin Hu. LightRetriever: A LLM-based Text Retrieval Architecture with Extremely Faster Query Inference. ICLR, 2026. [arXiv:2505.12260](#).
- [pyNIFE] Stephan Tulkens. pyNIFE: Nearly Inference Free Embeddings in Python. Software, 2025. [Repository](#).
- [Model2Vec] MinishLab. Model2Vec: Fast State-of-the-Art Static Embeddings. Software. [Repository](#).
- [StaticEmb] Tom Aarsen. Train 400x faster Static Embedding Models with Sentence Transformers. Hugging Face, 15 January 2025. [Blog post](#).
- [Cho and Hariharan 2019] Jang Hyun Cho and Bharath Hariharan. On the Efficacy of Knowledge Distillation. ICCV, 2019. [Proceedings](#).
- [PROD] Zhenghao Lin, Yeyun Gong, Xiao Liu, et al. PROD: Progressive Distillation for Dense Retrieval. WWW, 2023. [arXiv:2209.13335](#).
- [Arabzadeh et al.] Negar Arabzadeh, Xinyi Yan, and Charles L. A. Clarke. Predicting Efficiency/Effectiveness Trade-offs for Dense vs. Sparse Retrieval Strategy Selection. CIKM, 2021. [arXiv:2109.10739](#).
- [QPP] Guglielmo Faggioli, Thibault Formal, Stefano Marchesin, Stéphane Clinchant, Nicola Ferro, and Benjamin Piwowarski. Query Performance Prediction for Neural IR: Are We There Yet? 2023. [arXiv:2302.09947](#).
- [BCT] Yantao Shen, Yuanjun Xiong, Wei Xia, and Stefano Soatto. Towards Backward-Compatible Representation Learning. CVPR, 2020. [Proceedings](#).
- [FCT] Vivek Ramanujan, Pavan Kumar Anasosalu Vasu, Ali Farhadi, Oncel Tuzel, and Hadi Pouransari. Forward Compatible Training for Large-Scale Embedding Retrieval Systems. CVPR, 2022. [arXiv:2112.02805](#).
- [OOD-DiskANN] Shikhar Jaiswal, Ravishankar Krishnaswamy, Ankit Garg, Harsha Vardhan Simhadri, and Sheshansh Agrawal. OOD-DiskANN: Efficient and Scalable Graph ANNS for Out-of-Distribution Queries. 2022. [arXiv:2211.12850](#).
- [RoarGraph] Meng Chen, Kai Zhang, Zhenying He, Yinan Jing, and X. Sean Wang. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. PVLDB 17(11): 2735-2749, 2024. [Paper](#).
- [QA-Cos] Daehun Nyang. Beyond Hamming: Query-Aware Decoding of Binary Cosine Sketches. ICML 2026, PMLR 306: 94126-94143. [Proceedings](#).
- [Qdrant 1.19.1] Qdrant contributors. Binary query encoding and vector-index search implementation, v1.19.1. [Tagged source](#). Exact source paths and semantics are recorded in `m15/REVIEWS/2026-10-01-quantization-semantics.md`.